

PARADIGMAS DE PROGRAMACIÓN
CURSO SUAREZ

Trabajo Práctico 1

Programación Orientada a Objetos

× ×

23 de abril de 2026

Alexander Cruz

109044

acruz@fi.uba.ar

Índice

1. Introducción	3
2. Objetivo	3
3. Consigna	3
4. Alcance	3
5. Detalles de Implementación	4
5.1. Ejemplos de Polimorfismo	4
5.2. Mecanismos utilizados	5
6. Diagrama de Clases	6
7. Diagrama de Secuencias	7
7.1. El equipo está empatando, se actualiza el rendimiento	7
7.2. Al agregar un JugadorTitular con habilidad inconsistente se lanza una excepción .	8
8. Cobertura	8

1. Introducción

El presente informe detalla la solución desarrollada para el Trabajo Práctico 1, que tiene como objetivo aplicar los conceptos fundamentales de la programación orientada a objetos (POO) en el lenguaje Pharo. El problema, que consiste en el desarrollo de un sistema de recomendación de futbolistas, fue resuelto utilizando la metodología Test-Driven Development (TDD), un enfoque que garantiza la robustez y funcionalidad del código a través de pruebas unitarias.

La solución se centra en la construcción de un modelo de clases que se adhiere a los principios de la POO, como el encapsulamiento y, de manera crucial, el polimorfismo. Se ha diseñado una arquitectura que permite manejar la lógica de cálculo del valor de los futbolistas y los distintos criterios de búsqueda de manera flexible y extensible, evitando el uso de estructuras de control rígidas.

Adicionalmente, se ha utilizado la herramienta Mutalk para llevar a cabo una prueba de mutación. Este proceso aseguró la calidad de las pruebas unitarias creadas por el alumno, garantizando que el código no solo funciona, sino que está correctamente validado contra posibles errores. El informe incluye los diagramas de clases y de secuencia que documentan el diseño, junto con una explicación detallada de la implementación y los mecanismos de polimorfismo utilizados.

2. Objetivo

Este es un trabajo práctico individual que tiene como objetivo aplicar los conceptos del paradigma de la orientación a objetos vistos hasta ahora en el curso mediante la resolución de un problema con Pharo.

3. Consigna

Se deberá desarrollar un modelo de clases utilizando la metodología TDD. Las especificaciones son un archivo de pruebas SUnit (.st) adjunto a este enunciado y están dadas en forma de pruebas de casos de uso. Se espera que el alumno genere una solución que permita ejecutar exitosamente las pruebas especificadas manteniendo una alta cobertura y evitando la supervivencia de mutantes. Revisar el material Calidad de nuestras pruebas.

4. Alcance

Se deberá desarrollar el modelo completo de la aplicación, sin incluir la interfaz gráfica. Para esto se especifica una API que está descrita por un conjunto de casos de uso. Este conjunto de pruebas está descrito por la clase dada como enunciado.

Para el manejo de errores y situaciones de excepción, se deberán desarrollar y utilizar las excepciones que se crean convenientes, más allá de que en la prueba de integración provistas no necesariamente se manejan excepciones.

Deberán entregarse:

- todas las clases con sus métodos que resuelven el problema, organizados en una o más categorías según criterio del alumno;
- conjunto de pruebas que garanticen la no supervivencia de mutantes;
- conjunto de pruebas que garanticen la no supervivencia de mutantes;
- un archivo ".txt" que contenga la pieza de código que permite ejecutar los test de mutación (descrito aquí: pruebas de mutación);

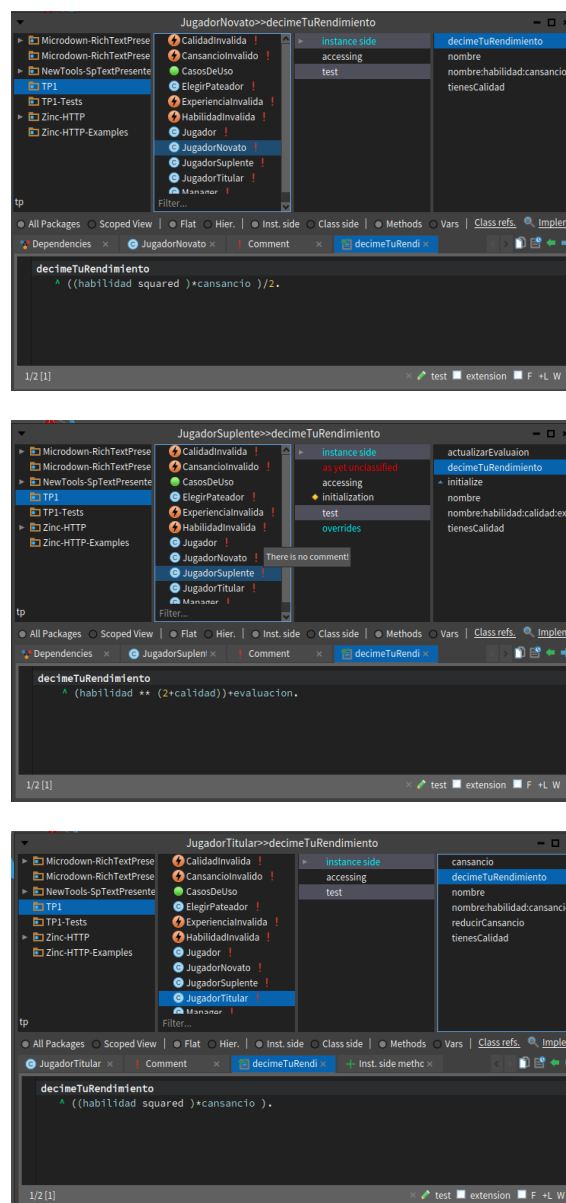
documentación completa del diseño de clases incluyendo diagramas UML de clases y secuencia (cantidades variables acordes al diseño presentado). Se considerarán entregas válidas a aquellas que contengan las partes anteriores. Las que no tengan alguna de ellas no recibirán corrección.

5. Detalles de Implementación

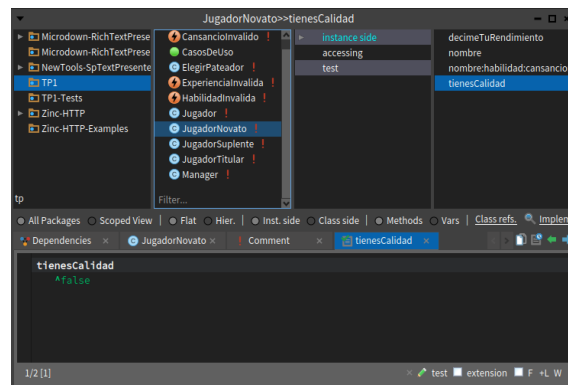
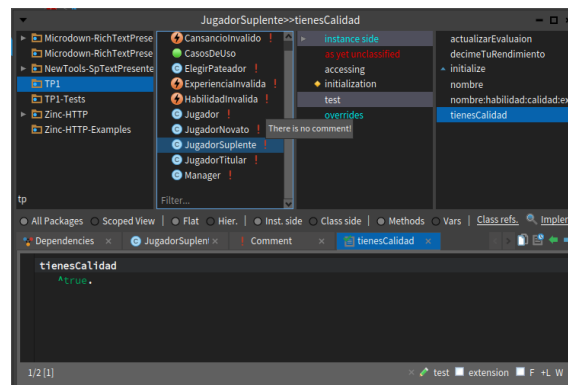
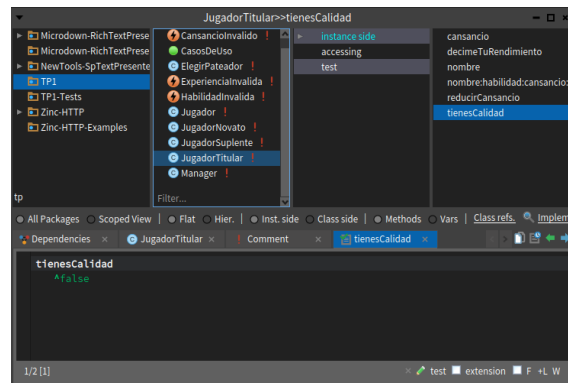
5.1. Ejemplos de Polimorfismo

El Polimorfismo, una de las piedras angulares de la Programación Orientada a Objetos (POO).

EL primer ejemplo de Polimorfismo lo podemos observar en tres clases distintas (JugadorTitular, JugadorSuplente y JugadorNovato), cada tiene su propia implementacion del metodo decimeTuRendimiento().

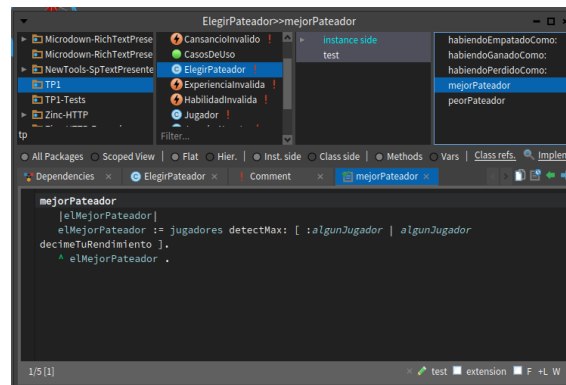


Como segundo ejemplo de la implementacion de polimorfismo, incluye nuevamente a las 3 clases mencionadas anteriormente, pero en esta ocasión la implementacion de cada una consiste en agregar un true/false donde corresponda.



5.2. Mecanismos utilizados

El objeto ElegirPateador como cliente no tiene idea de la implementación del método `decimeTuRendimiento()` solo sabe que puede pedirlo, esto es un gran mecanismo de encapsulamiento el cual nos permite agregar varios objetos que pueden responder al mismo mensaje, podemos observar un ejemplo concreto en la siguiente imagen.



6. Diagrama de Clases

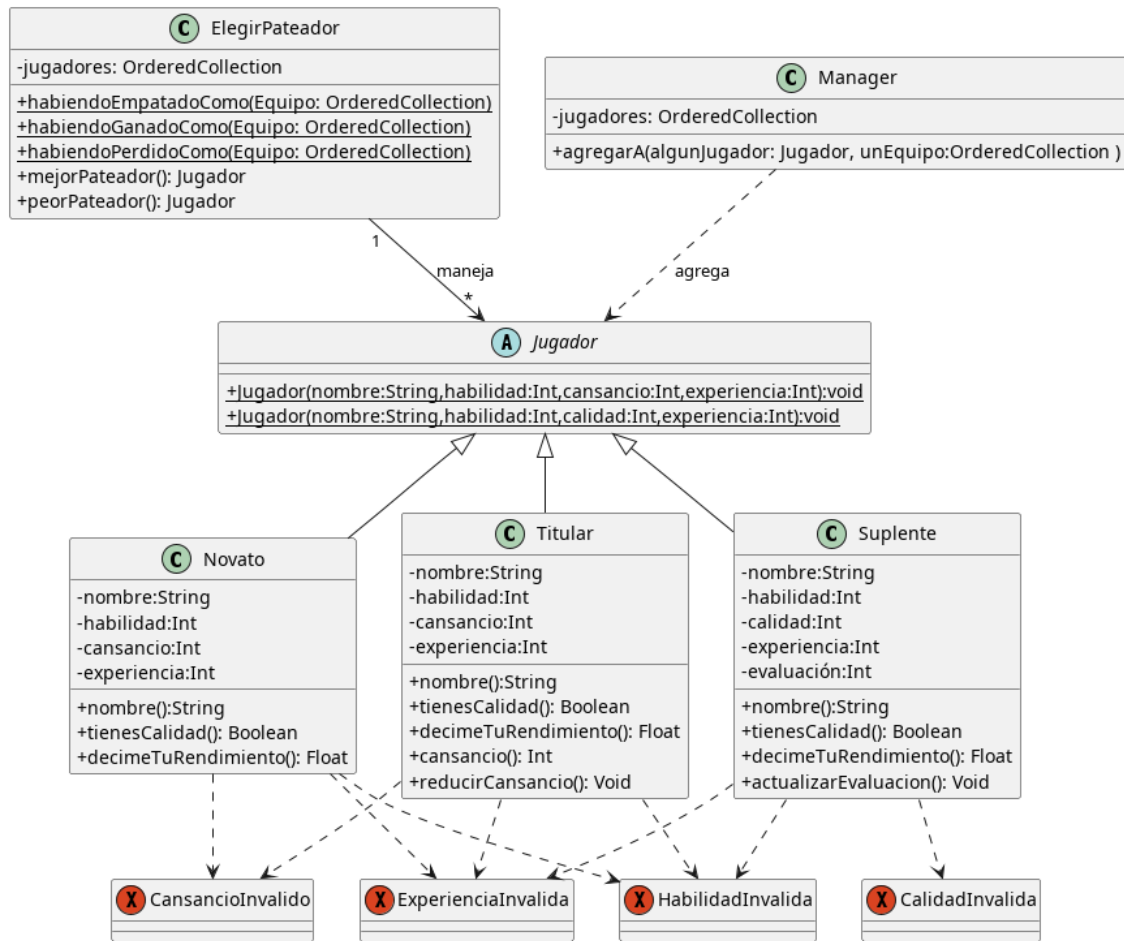
El diagrama representa la arquitectura del sistema para la gestión y selección de pateadores de penales. El diseño se centra en la flexibilidad y el aprovechamiento del polimorfismo para manejar diferentes estados de juego y tipos de jugadores.

Estructura de Clases y Jerarquía

- **Jugador:** Define un contrato básico como una clase abstracta de la cual los únicos métodos que usaran son constructores para jugadores que tengan cansancio y calidad.
- **Subclases Especializadas (Novato, Titular, Suplente):** Cada una encapsula su propia lógica de rendimiento. Por ejemplo, el JugadorNovato divide su potencial por dos debido a su falta de experiencia, mientras que el JugadorSuplente basa su rendimiento en una potencia que incluye su calidad y una evaluación variable.
- **Clase ElegirPateador:** Es la encargada de la lógica de negocio para decidir quién ejecuta el penal. Utiliza constructores con nombre (ej. `habiendoGanadoComo:`) para configurar el estado de los jugadores antes de la elección.

Relaciones Y Comportamientos Estáticos

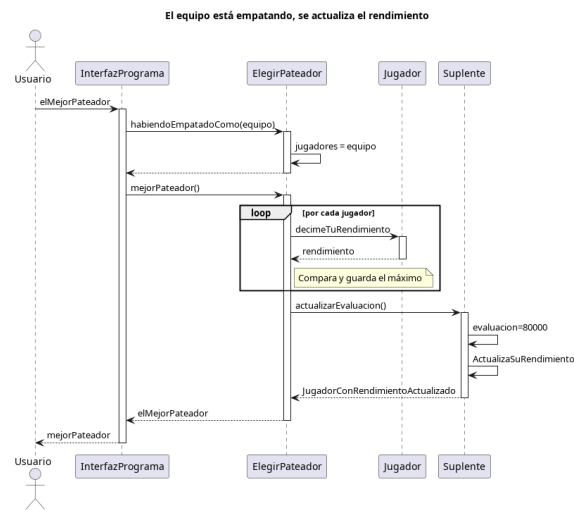
- **Polimorfismo:** Se observa en los métodos `decimeTuRendimiento` y `tienesCalidad`. Esto permite que `ElegirPateador` pueda iterar sobre una colección mixta de jugadores y obtener sus valores de rendimiento sin conocer su clase específica.
- **Manejo de Errores:** El diagrama muestra una relación de dependencia hacia las clases de excepción (`HabilidadInvalida`, `CansancioInvalido`, etc.). Estas son señaladas durante la instanciación de los jugadores para garantizar que el sistema siempre opere con datos válidos.
- **Asociación:** La clase `ElegirPateador` mantiene una relación de agregación con una colección de objetos que heredan de `Jugador`, permitiendo la selección dinámica del mejor o peor perfil para el tiro.



7. Diagrama de Secuencias

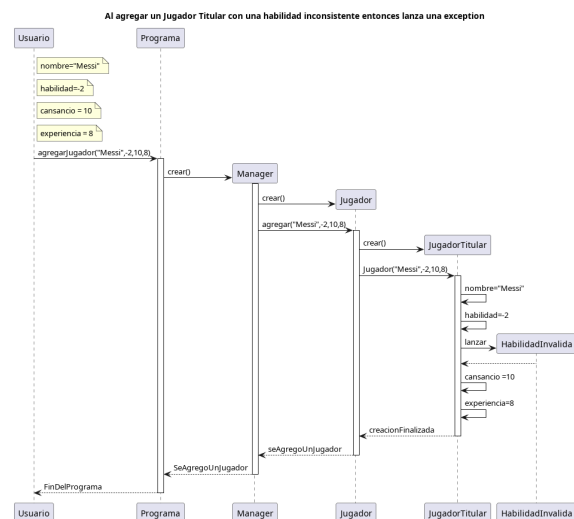
7.1. El equipo está empatando, se actualiza el rendimiento

Este diagrama modela el escenario en el que el sistema debe encontrar al mejor pateador cuando el equipo empató un partido. El flujo comienza cuando el Usuario solicita el mejor pateador a la InterfazPrograma. La secuencia interesante ocurre en mejorPateador(): se itera sobre cada jugador del equipo (loop) enviándole el mensaje decimeTuRendimiento. Internamente, ElegirPateador compara y retiene al jugador con el rendimiento máximo usando detectMax:. Una vez identificado el mejor pateador, si el jugador es un Suplente, se le envía actualizarEvaluacion(), que fija su evaluacion en 80000 y luego ejecuta ActualizaSuRendimiento. Finalmente el objeto resultante es devuelto al Usuario.



7.2. Al agregar un JugadorTitular con habilidad inconsistente se lanza una excepción

Este diagrama ilustra el escenario de error al intentar crear un JugadorTitular con una habilidad inválida (valor negativo). El Usuario provee los datos nombre="Messi", habilidad=-2, cansancio=10, experiencia=8. El flujo pasa por el Programa, que delega en el Manager la creación. El Manager crea un Jugador y le envía el mensaje agregar("Messi",-2,10,8). Jugador, a su vez, instancia un JugadorTitular y le envía los datos uno a uno. La parte más relevante ocurre cuando JugadorTitular recibe habilidad=-2: detecta que el valor es inconsistente y lanza la excepción HabilidadInvalida. A pesar del error, la secuencia continúa asignando cansancio y experiencia, y el objeto retorna creacionFinalizada, lo que evidencia que la excepción se lanza pero no interrumpe el flujo completo de inicialización en esta implementación.



8. Cobertura

Se logro obtener una buena cobertura de pruebas cercanas al 90 por ciento.

Not Covered Code (87% Code Coverage) [4]

ElegirPateador	habiendoGanadoComo:	test	TP1
ElegirPateador class	habiendoGanadoComo:	test	TP1
ElegirPateador	habiendoPerdidoComo:	test	TP1
ElegirPateador class	habiendoPerdidoComo:	test	TP1

Filter...

Current image ▾ Flat Hier. | Browse Class refs. Implementors Senders

habiendoGanac x

```
habiendoGanadoComo: unEquipoColeccion

jugadores := unEquipoColeccion collect: [ :unJugador |
    (unJugador tienesCalidad) ifTrue: [ unJugador actualizarEvaluacion ]
    ifFalse: [ unJugador ]
]
```

1/6 [1] test extension F +L W